

Informe Triggers Módulo 5

Gestión de Suministros

Versión: 1.1

20 de mayo de 2026

Juan David Peña Roldán

Universidad Surcolombiana

Neiva, Huila

2026

TGR-M05-01:

trg_consumo_alimento_inmutable_vali dado

Descripción	Impide cualquier operación UPDATE sobre un registro de consumo de alimento en estado VALIDADO. Amplía la protección de campos respecto a la versión anterior incluyendo los nuevos campos fecha_consumo, hora_suministro y costo_unitario que ahora existen en la tabla. BEFORE UPDATE sobre registros_consumo_alimentos. Bloquea edición cuando el estado es VALIDADO y se intenta mantener en VALIDADO. Permite únicamente la transición a ANULADO, pero protegiendo todos los campos de negocio incluyendo los nuevos.
Aplicado a	RF-75 — Registro de Consumo de Alimentos
Restricción origen RF	<i>RF-75, Restricciones, punto 5:</i> "Un registro en estado VALIDADO es inmutable. No puede ser editado. Solo puede ser ANULADO por el Veterinario o Administrador con campo justificacion_anulacion obligatorio (mínimo 50 caracteres)." <i>RF-75, Flujo alterno E5:</i> "HTTP 405: Method Not Allowed — El registro está en estado VALIDADO y no puede ser modificado."
Justificación	La inmutabilidad del registro validado es la última línea de defensa de integridad. Con el nuevo esquema hay más campos de negocio que proteger durante la transición a ANULADO; si no se actualiza el trigger, un UPDATE de anulación podría colateralmente alterar fecha_consumo o costo_unitario sin ser detectado.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_trg_consumo_alimento_inmutable_validado()
RETURNS TRIGGER AS $$
BEGIN
    -- Bloquear cualquier edición si el registro está VALIDADO
    -- y se intenta mantener en VALIDADO
    IF OLD.estado_registro = 'VALIDADO' AND NEW.estado_registro =
'VALIDADO' THEN
        RAISE EXCEPTION
            'INMUTABILIDAD_VIOLADA: El registro de consumo con id
[%] está en '
            'estado VALIDADO y no puede ser modificado. Solo se
permite la '
            'transición a ANULADO. (RF-75 Restricción 5)',
            OLD.id_consumo_alimento
        USING ERRCODE = 'P0001';
    END IF;

    -- Durante la transición VALIDADO → ANULADO, proteger todos
los
    -- campos de negocio, incluyendo los nuevos del esquema
actualizado
    IF OLD.estado_registro = 'VALIDADO' AND NEW.estado_registro =
'ANULADO' THEN
        IF NEW.id_activo_biologico      IS DISTINCT FROM
OLD.id_activo_biologico      OR
        NEW.id_tipo_alimento           IS DISTINCT FROM
OLD.id_tipo_alimento         OR
        NEW.tipo_alimento              IS DISTINCT FROM
OLD.tipo_alimento           OR
        NEW.cantidad_suministrada      IS DISTINCT FROM
OLD.cantidad_suministrada   OR
        NEW.costo_total               IS DISTINCT FROM
OLD.costo_total             OR
        NEW.costo_unitario            IS DISTINCT FROM
OLD.costo_unitario          OR
        NEW.fecha_consumo              IS DISTINCT FROM
OLD.fecha_consumo           OR
        NEW.hora_suministro            IS DISTINCT FROM
OLD.hora_suministro         OR
        NEW.fecha_inicio_periodo      IS DISTINCT FROM

```

```

OLD.fecha_inicio_periodo OR
    NEW.fecha_fin_periodo IS DISTINCT FROM
OLD.fecha_fin_periodo OR
    NEW.consumo_por_individuo_kg IS DISTINCT FROM
OLD.consumo_por_individuo_kg
    THEN
        RAISE EXCEPTION
            'INMUTABILIDAD_VIOLADA: Durante la anulación del
registro de '
            'consumo [%] no se pueden modificar campos de
negocio. Solo '
            'se permite cambiar el estado y los campos propios
de la '
            'anulación (justificacion_anulacion,
fecha_hora_anulacion). '
            '(RF-75 Restricción 5)',
            OLD.id_consumo_alimeto
        USING ERRCODE = 'P0001';
    END IF;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_consumo_alimento_inmutable_validado
BEFORE UPDATE ON modulo5.registros_consumo_alimentos
FOR EACH ROW
EXECUTE FUNCTION
modulo5.fn_trg_consumo_alimento_inmutable_validado();

```

TGR-M05-02:

trg_medicamento_inmutable_validado

Descripción	Impide cualquier UPDATE sobre un registro de medicamento en estado VALIDADO y bloquea la re-anulación de registros ya ANULADO . Amplía la protección de campos respecto a la versión anterior incluyendo
-------------	--

	<code>hora_aplicacion</code> y <code>motivo_aplicacion</code> que ahora existen en la tabla. BEFORE UPDATE sobre <code>registros_medicamentos</code>. Bloquea re-anulación, bloquea edición de registros VALIDADO, y protege todos los campos de negocio incluyendo los nuevos durante la transición a ANULADO.
Aplicado a	RF-76 — Registro de Aplicación de Medicamentos
Restricción origen RF	<i>RF-76, Restricciones, punto 5:</i> "Un registro de aplicación en estado VALIDADO es inmutable. Solo puede ser anulado por el Veterinario o Administrador con <code>justificacion_anulacion</code> obligatoria (mínimo 20 caracteres)." <i>RF-76, Flujo alterno E8:</i> "HTTP 409: Conflict — El registro ya se encuentra en estado ANULADO . No es posible anular un registro que ya fue anulado previamente."
Justificación	Con los nuevos campos <code>hora_aplicacion</code> y <code>motivo_aplicacion</code> en la tabla, la protección debe extenderse para que una operación de anulación no pueda alterar datos clínicos. La hora de aplicación y el motivo son parte de la trazabilidad sanitaria exigida por el ICA.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_trg_medimento_inmutable_validado()
RETURNS TRIGGER AS $$
BEGIN
    -- Bloquear cualquier UPDATE sobre un registro ya ANULADO
    IF OLD.estado_registro = 'ANULADO' THEN
        RAISE EXCEPTION
            'RE_ANULACION_INVALIDA: El registro de medicamento con id
[%] ya '
            'se encuentra en estado ANULADO. No es posible anular un
registro '
            'que ya fue anulado previamente. (RF-76 Flujo alterno E8)',
        OLD.id_registro_medimento

```

```

        USING ERRCODE = 'P0001';
    END IF;

    -- Bloquear edición si el registro está VALIDADO y se mantiene
VALIDADO
    IF OLD.estado_registro = 'VALIDADO' AND NEW.estado_registro =
'VALIDADO' THEN
        RAISE EXCEPTION
            'INMUTABILIDAD_VIOLADA: El registro de medicamento con id
[%] está '
                'en estado VALIDADO y no puede ser modificado. Solo se
permite la '
                'transición a ANULADO. (RF-76 Restricción 5)',
            OLD.id_registro_medamento
        USING ERRCODE = 'P0001';
    END IF;

    -- Durante la transición VALIDADO → ANULADO, proteger todos los
    -- campos de negocio incluyendo los nuevos del esquema actualizado
    IF OLD.estado_registro = 'VALIDADO' AND NEW.estado_registro =
'ANULADO' THEN
        IF NEW.id_activo_biologico          IS DISTINCT FROM
OLD.id_activo_biologico          OR
        NEW.nombre_medamento              IS DISTINCT FROM
OLD.nombre_medamento            OR
        NEW.cantidad                      IS DISTINCT FROM OLD.cantidad
    OR
        NEW.fecha_aplicacion              IS DISTINCT FROM
OLD.fecha_aplicacion             OR
        NEW.hora_aplicacion                IS DISTINCT FROM
OLD.hora_aplicacion              OR
        NEW.costo_unitario_medamento     IS DISTINCT FROM
OLD.costo_unitario_medamento    OR
        NEW.costo_total_medamento        IS DISTINCT FROM
OLD.costo_total_medamento       OR
        NEW.motivo_aplicacion              IS DISTINCT FROM
OLD.motivo_aplicacion            OR
        NEW.via_aplicacion                 IS DISTINCT FROM
OLD.via_aplicacion               OR
        NEW.periodo_retiro_dias            IS DISTINCT FROM
OLD.periodo_retiro_dias          OR
        NEW.fecha_fin_retiro               IS DISTINCT FROM
OLD.fecha_fin_retiro             OR
        NEW.dosis_por_individuo            IS DISTINCT FROM
OLD.dosis_por_individuo
    THEN

```

```

        RAISE EXCEPTION
        'INMUTABILIDAD_VIOLADA: Durante la anulación del
registro de '
        'medicamento [%] no se pueden modificar campos de
negocio. Solo '
        'se permite cambiar el estado y los campos propios de la
',
        'anulación (justificacion_anulacion,
fecha_hora_anulacion). '
        '(RF-76 Restricción 5)',
        OLD.id_registro_medicamento
        USING ERRCODE = 'P0001';
    END IF;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_medicamento_inmutable_validado
BEFORE UPDATE ON modulo5.registros_medicamentos
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_medicamento_inmutable_validado();

```

TGR-M05-03:

trg_calcular_costo_total_consumo

Descripción	
	Calcula automáticamente costo_total en registros_consumo_alimentos al INSERT. Con el nuevo esquema la tabla ya tiene costo_unitario como campo propio, por lo que la lógica de resolución cambia: si NEW.costo_unitario viene informado se usa directamente; si no, se consulta el catálogo. Adicionalmente absorbe la validación de estado activo del tipo de alimento, eliminando la necesidad del antiguo TGR-M05-15 como trigger independiente. BEFORE INSERT sobre registros_consumo_alimentos.

	Valida cantidad positiva, verifica estado activo del tipo de alimento, resuelve costo_unitario (propio o del catálogo) y calcula costo_total .
Aplicado a	RF-75 — Registro de Consumo de Alimentos
Restricción origen RF	<i>RF-75, Proceso paso 8.1:</i> "El sistema calcula: $\text{costo_total_alimento} = \text{cantidad_alimento} \times \text{costo_unitario}$." <i>RF-75, Salida punto 3:</i> "Costo total de la alimentación calculado y persistido." <i>RF-75, Restricciones punto 1:</i> "cantidad_alimento_kg debe ser mayor a cero." <i>RF-75, Proceso paso 2:</i> "El sistema valida que el activo y los elementos del catálogo estén en estado ACTIVO."
Justificación	El nuevo campo costo_unitario en la fila permite que el backend envíe el precio directamente (por ejemplo, cuando viene de una negociación puntual distinta al catálogo). El trigger debe respetar ese valor si viene, y solo consultar el catálogo como fallback. Esto mantiene el cálculo determinístico y la BD como fuente de verdad.

```

CREATE OR REPLACE FUNCTION modulo5.fn_trg_calcular_costo_total_consumo()
RETURNS TRIGGER AS $$
DECLARE
    v_costo_unitario modulo5.tipos_alimentos.costo_unitario%TYPE;
    v_estado_alimento modulo5.enum_tipo_alimento_estado;
BEGIN
    -- Validar cantidad positiva
    IF NEW.cantidad_suministrada <= 0 THEN
        RAISE EXCEPTION
            'CANTIDAD_INVALIDA: La cantidad suministrada debe ser mayor
a cero. '
            'Valor recibido: [%]. (RF-75 Restricción 1)',
            NEW.cantidad_suministrada
        USING ERRCODE = 'P0001';
    END IF;

    -- Verificar existencia y estado activo del tipo de alimento

```



```

-- (absorbe la lógica del antiguo TGR-M05-15)
SELECT estado, costo_unitario
INTO v_estado_alimento, v_costo_unitario
FROM modulo5.tipos_alimentos
WHERE id_tipo_elemento = NEW.id_tipo_alimento;

IF NOT FOUND THEN
    RAISE EXCEPTION
        'TIPO_ALIMENTO_NO_ENCONTRADO: El tipo de alimento con id [%]
no '
        'existe en el catálogo. Verifique el identificador. '
        '(RF-75 Proceso paso 2)',
        NEW.id_tipo_alimento
    USING ERRCODE = 'P0001';
END IF;

IF v_estado_alimento != 'ACTIVO' THEN
    RAISE EXCEPTION
        'TIPO_ALIMENTO_INACTIVO: El tipo de alimento con id [%] está
en '
        'estado [%] y no puede usarse para nuevos registros de
consumo. '
        'Solo los tipos en estado ACTIVO son válidos. '
        '(RF-75 Proceso paso 2)',
        NEW.id_tipo_alimento,
        v_estado_alimento
    USING ERRCODE = 'P0001';
END IF;

-- Resolver costo_unitario: usar el informado en la fila si viene,
-- de lo contrario usar el del catálogo como fuente de verdad
IF NEW.costo_unitario IS NOT NULL AND NEW.costo_unitario > 0 THEN
    v_costo_unitario := NEW.costo_unitario;
ELSE
    -- Asignar el del catálogo al campo de la fila para persistirlo
    NEW.costo_unitario := v_costo_unitario;
END IF;

-- Calcular y asignar costo_total como fuente de verdad en BD
NEW.costo_total := NEW.cantidad_suministrada * v_costo_unitario;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_calcular_costo_total_consumo

```

```
BEFORE INSERT ON modulo5.registros_consumo_alimentos
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_calcular_costo_total_consumo();
```

TGR-M05-04:

trg_calculo_costo_total_medicamento

Descripción	Calcula automáticamente el campo costo_total_medicamento al momento del INSERT en registros_medicamentos, como producto de cantidad × costo_unitario_medicamento. Se dispara en BEFORE INSERT sobre registros_medicamentos. Calcula costo_total_medicamento = cantidad × costo_unitario_medicamento. Valida que cantidad > 0 y que costo_unitario_medicamento > 0.
Aplicado a	RF-76 — Registro de Aplicación de Medicamentos
Restricción origen RF	<i>RF-76, Proceso, paso 8a:</i> "El sistema calcula: costo_total_medicacion = costo_unitario × dosis_aplicada" <i>RF-76, Salida, punto 4:</i> "Costo total de la medicación calculado y persistido."
Justificación	Mismo principio que TGR-M05-03. Es un cálculo matemático simple y determinístico. Garantiza que el campo derivado sea siempre consistente con sus componentes, independientemente del canal de inserción.

```
CREATE OR REPLACE FUNCTION modulo5.fn_calculo_costo_total_medicamento()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Validar cantidad positiva
    IF NEW.cantidad <= 0 THEN
        RAISE EXCEPTION
```

```

        'CANTIDAD_INVALIDA: La dosis aplicada debe ser mayor a cero.
Valor recibido: %',
        NEW.cantidad
        USING ERRCODE = 'P0001';
    END IF;

    -- Validar costo unitario positivo
    IF NEW.costo_unitario_medimento <= 0 THEN
        RAISE EXCEPTION
        'COSTO_INVALIDO: El costo unitario del medicamento debe ser
mayor a cero. Valor recibido: %',
        NEW.costo_unitario_medimento
        USING ERRCODE = 'P0001';
    END IF;

    -- Calcular y asignar el costo total (fuente de verdad en BD)
    NEW.costo_total_medimento := NEW.cantidad *
NEW.costo_unitario_medimento;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_calculo_costo_total_medimento
    BEFORE INSERT ON modulo5.registros_medamentos
    FOR EACH ROW
    EXECUTE FUNCTION modulo5.fn_calculo_costo_total_medimento();

```

TGR-M05-05:

trg_proteccion_delete_consumo_alimento

Descripción	Bloquea cualquier intento de eliminación física (DELETE) sobre registros de consumo de alimentos, independientemente de su estado. Se dispara en BEFORE DELETE sobre registros_consumo_alimentos para cada fila. Siempre lanza una excepción, bloqueando la operación sin excepción
-------------	--

	posible.
Aplicado a	RF-75 — Registro de Consumo de Alimentos / RF-78 — Acumulación de Inversión
Restricción origen RF	<i>RF-75, Restricciones, punto 6:</i> "Un registro ANULADO no puede ser reactivado. Si se necesita reemplazarlo, se debe crear un nuevo registro." (implica que los registros nunca se eliminan) <i>RF-78, Restricciones, punto 6:</i> "Los registros de suministros son append-only. No se permiten DELETE ni UPDATE sobre registros históricos." <i>RF-75, Postcondiciones, punto 4:</i> "Un registro ANULADO queda excluido de todos los cálculos pero permanece visible en el historial."
Justificación	La política append-only es una garantía de trazabilidad que debe imponerse a nivel de motor de base de datos. Ninguna lógica de aplicación, migración mal escrita o acceso directo debe poder borrar el historial de suministros.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_proteccion_delete_consumo_alimento()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    RAISE EXCEPTION
        'OPERACION_NO_PERMITIDA: Los registros de consumo de alimentos
son append-only. No se permite la eliminación del registro %. Para
invalidar un registro, utilice la operación de anulación.',
        OLD.id_consumo_alimento
    USING ERRCODE = 'P0001';
END;
$$;

CREATE TRIGGER trg_proteccion_delete_consumo_alimento
BEFORE DELETE ON modulo5.registros_consumo_alimentos
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_proteccion_delete_consumo_alimento();

```

TGR-M05-06:

trg_proteccion_delete_medicamento

Descripción	Bloquea cualquier intento de eliminación física (DELETE) sobre registros de aplicación de medicamentos. Se dispara en BEFORE DELETE sobre registros_medicamentos. Siempre lanza una excepción.
Aplicado a	RF-76 — Registro de Aplicación de Medicamentos / RF-78 — Acumulación de Inversión
Restricción origen RF	<i>RF-78, Restricciones, punto 6:</i> "Los registros de suministros son append-only. Las correcciones generan un registro de ajuste (tipo_operacion = CORRECCION). No se permiten DELETE ni UPDATE sobre registros históricos." <i>RF-76, Restricciones, punto 5:</i> El historial de tratamientos debe conservarse íntegro para trazabilidad sanitaria.
Justificación	Misma razón que TGR-M05-05. La integridad del historial farmacológico es crítica para auditorías del ICA y trazabilidad NIC 41. No puede depender de restricciones de aplicación.

```
CREATE OR REPLACE FUNCTION modulo5.fn_proteccion_delete_medicamento()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    RAISE EXCEPTION
        'OPERACION_NO_PERMITIDA: Los registros de aplicación de
        medicamentos son append-only. No se permite la eliminación del registro
        %. Para invalidar un registro, utilice la operación de anulación.',
        OLD.id_registro_medicamento
    USING ERRCODE = 'P0001';
END;
$$;
```

```
CREATE TRIGGER trg_proteccion_delete_medicamento
  BEFORE DELETE ON modulo5.registros_medicamentos
  FOR EACH ROW
  EXECUTE FUNCTION modulo5.fn_proteccion_delete_medicamento();
```

TGR-M05-07:

trg_proteccion_delete_reporte_gastos

Descripción	Bloquea la eliminación física de reportes de gastos acumulados una vez generados. Se dispara en BEFORE DELETE sobre reporte_gastos_acumulados. Siempre lanza una excepción bloqueando el DELETE.
Aplicado a	RF-77 — Reporte de Gastos Acumulados
Restricción origen RF	<i>RF-77, Postcondiciones, punto 2:</i> "El documento exportado queda temporalmente disponible para descarga por el usuario durante 24 horas." (la expiración aplica a archivos, no al registro del reporte) <i>RF-77, Requerimientos no funcionales — Historial de reportes:</i> "Los parámetros guardados no tienen fecha de expiración y permanecen disponibles en el historial del usuario indefinidamente."
Justificación	El registro del reporte en BD debe ser permanente para el historial del usuario. La expiración de archivos exportados es responsabilidad del backend/storage, no implica eliminar el registro de la tabla.

```
CREATE OR REPLACE FUNCTION modulo5.fn_proteccion_delete_reporte_gastos()
  RETURNS TRIGGER
  LANGUAGE plpgsql
  AS $$
  BEGIN
    RAISE EXCEPTION
      'OPERACION_NO_PERMITIDA: Los reportes de gastos acumulados no
      pueden eliminarse. El registro con id % debe conservarse en el historial
      de reportes del usuario de forma indefinida.',
```

```

        OLD.id_reporte_gasto_acumulado
        USING ERRCODE = 'P0001';
END;
$$;

CREATE TRIGGER trg_proteccion_delete_reporte_gastos
    BEFORE DELETE ON modulo5.reporte_gastos_acumulados
    FOR EACH ROW
    EXECUTE FUNCTION modulo5.fn_proteccion_delete_reporte_gastos();

```

TGR-M05-08:

trg_proteccion_delete_costos_productivos

Descripción	Bloquea la eliminación de registros de costos productivos una vez consolidados, garantizando la inmutabilidad del acumulado del ciclo. Se dispara en BEFORE DELETE sobre costos_productivos. Siempre lanza una excepción.
Aplicado a	RF-78 — Acumulación de Inversión
Restricción origen RF	<i>RF-78, Restricciones, punto 6:</i> "Los registros de suministros son append-only. Las correcciones generan un registro de ajuste (tipo_operacion = CORRECCION) con referencia al registro original (id_registro_original) y motivo_correccion obligatorio. No se permiten DELETE ni UPDATE sobre registros históricos." <i>RF-78, Restricciones, punto 2:</i> "Una vez cerrado el ciclo por RF-41, el monto_inversion_total_ciclo es inmutable. No se admiten nuevos registros ni modificaciones sobre ese ciclo."
Justificación	La inmutabilidad del acumulado de ciclo es un requisito NIC 41. Debe ser garantizada a nivel de motor.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_proteccion_delete_costos_productivos()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    RAISE EXCEPTION
        'OPERACION_NO_PERMITIDA: Los registros de costos productivos son
        append-only bajo política NIC 41. No se permite la eliminación del
        registro %. Las correcciones deben realizarse mediante registros de
        ajuste con tipo_operacion = CORRECCION.',
        OLD.id_costo_productivo
    USING ERRCODE = 'P0001';
END;
$$;

CREATE TRIGGER trg_proteccion_delete_costos_productivos
    BEFORE DELETE ON modulo5.costos_productivos
    FOR EACH ROW
    EXECUTE FUNCTION modulo5.fn_proteccion_delete_costos_productivos();

```

TGR-M05-09:

trg_consumo_alimento_fecha_no_futura

Descripción	Valida que la fecha de consumo de alimento no sea posterior a la fecha actual del servidor. Con el nuevo esquema la tabla tiene fecha_consumo (tipo date) como campo canónico dedicado, además del fecha_inicio_periodo (tipo timestamp) que existía antes. El trigger ahora valida ambos campos con sus respectivos tipos. BEFORE INSERT sobre registros_consumo_alimentos. Valida fecha_consumo > CURRENT_DATE (campo canónico nuevo) y como segunda línea valida
-------------	---

	<code>fecha_inicio_periodo > NOW()</code> si está presente.
Aplicado a	RF-75 — Registro de Consumo de Alimentos
Restricción origen RF	<i>RF-75, Restricciones, punto 2:</i> "La <code>fecha_consumo</code> no puede ser posterior a la fecha actual del sistema (no se permiten registros futuros)." <i>RF-75, Flujo alterno E2:</i> "HTTP 400: Bad Request — La fecha de consumo [<code>fecha_consumo</code>] es posterior a la fecha actual del sistema."
Justificación	Con el nuevo campo <code>fecha_consumo</code> como campo canónico de tipo <code>date</code> , la validación debe aplicarse sobre él primero. El campo <code>fecha_inicio_periodo</code> es auxiliar para cálculos de período; <code>fecha_consumo</code> es el dato de negocio que representa cuándo ocurrió el suministro.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_trg_consumo_alimento_fecha_no_futura()
RETURNS TRIGGER AS $$
BEGIN
    -- Validar campo canónico fecha_consumo (tipo date, campo nuevo)
    IF NEW.fecha_consumo IS NOT NULL AND NEW.fecha_consumo >
CURRENT_DATE THEN
        RAISE EXCEPTION
            'FECHA_FUTURA_NO_PERMITIDA: La fecha de consumo [%] es
posterior '
            'a la fecha actual del sistema [%]. No se permiten registros
de '
            'consumo con fechas futuras. (RF-75 Restricción 2)',
            NEW.fecha_consumo,
            CURRENT_DATE
        USING ERRCODE = 'P0001';
    END IF;

    -- Validar también fecha_inicio_periodo si está presente (campo
auxiliar)
    IF NEW.fecha_inicio_periodo IS NOT NULL
        AND NEW.fecha_inicio_periodo > NOW() THEN
        RAISE EXCEPTION
            'FECHA_FUTURA_NO_PERMITIDA: La fecha_inicio_periodo [%] es

```

```

posterior '
        'a la fecha actual del sistema [%]. No se permiten registros
de '
        'consumo con fechas futuras. (RF-75 Restricción 2)',
        NEW.fecha_inicio_periodo,
        NOW()
        USING ERRCODE = 'P0001';
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_consumo_alimento_fecha_no_futura
BEFORE INSERT ON modulo5.registros_consumo_alimentos
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_consumo_alimento_fecha_no_futura();

```

TGR-M05-10:

trg_validacion_fecha_medicamento_n o_futura

Descripción	Valida que la fecha de aplicación del medicamento no sea posterior a la fecha actual del servidor. Se dispara en BEFORE INSERT sobre registros_medicamentos. Valida que fecha_aplicacion no sea mayor a CURRENT_DATE.
Aplicado a	RF-76 — Registro de Aplicación de Medicamentos
Restricción origen RF	RF-76, Flujo alterno E2: "fecha_aplicacion es posterior a la fecha actual del servidor. HTTP 400: Bad Request. Mensaje: 'La fecha de aplicación [FECHA_INGRESADA] es posterior a la fecha actual del sistema.'"
Justificación	Misma razón que TGR-M05-09. La BD es

	la última línea de defensa para integridad temporal de registros médicos.
--	---

```

CREATE OR REPLACE FUNCTION
modulo5.fn_validacion_fecha_medicamento_no_futura()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.fecha_aplicacion > CURRENT_DATE THEN
        RAISE EXCEPTION
            'FECHA_FUTURA_NO_PERMITIDA: La fecha de aplicación del
            medicamento (%) es posterior a la fecha actual del sistema (%). No se
            permiten registros con fechas futuras.',
            NEW.fecha_aplicacion,
            CURRENT_DATE
        USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validacion_fecha_medicamento_no_futura
BEFORE INSERT ON modulo5.registros_medicamentos
FOR EACH ROW
EXECUTE FUNCTION
modulo5.fn_validacion_fecha_medicamento_no_futura();

```

TGR-M05-11:

trg_auditoria_consumo_alimento

Descripción	Registra automáticamente en auditorias_suministros todo INSERT o UPDATE sobre registros_consumo_alimentos. Con el nuevo esquema de auditorias_suministros enriquecido con columnas RF-80 (id_activo_biologico, id_ciclo_productivo, costo_afectado, origen_precio,
-------------	---

	<code>clasificacion_registro</code>, <code>retencion_aplicable</code>, <code>fecha_emision</code>), el trigger ahora puebla esos campos extendidos. <code>AFTER INSERT OR UPDATE</code> sobre <code>registros_consumo_alimentos</code>. Puebla todos los campos base más los extendidos de RF-80. En INSERT registra <code>REGISTRO</code>; en UPDATE registra <code>ANULACION</code>.
Aplicado a	RF-75 — Registro de Consumo de Alimentos / RF-80 — Auditoría y Trazabilidad
Restricción origen RF	<i>RF-75, Criterios CA-7:</i> "El sistema registra en auditoría toda operación de registro, validación y anulación con usuario y timestamp." <i>RF-80, Restricciones, punto 8:</i> "Campos mínimos obligatorios para SUMINISTRO_REGISTRADO: <code>id_activo_biologico</code>, <code>id_ciclo_productivo</code>, <code>costo_afectado</code>, <code>origen_precio</code>, <code>id_usuario</code>." <i>RF-80, Descripción:</i> "Los eventos heredan el esquema base de RF-10 y se extienden con campos específicos del dominio de suministros."
Justificación	La tabla <code>auditorias_suministros</code> ahora tiene columnas extendidas alineadas con RF-80. Si el trigger no las puebla, los registros de auditoría quedarán con campos nulos que RF-80 declara como obligatorios para los eventos de tipo SUMINISTRO_REGISTRADO, dejando la auditoría incompleta.

```

CREATE OR REPLACE FUNCTION modulo5.fn_trg_auditoria_consumo_alimento()
RETURNS TRIGGER AS $$
DECLARE
    v_tipo_op      modulo5.enum_auditoria_suministro_tipo_operacion;
    v_datos_ant    json := NULL;
    v_datos_nue    json := NULL;
    v_costo        numeric(16,4) := NULL;
BEGIN
    IF TG_OP = 'INSERT' THEN

```

```

        v_tipo_op      := 'REGISTRO';
        v_datos_nue := row_to_json(NEW);
        v_costo        := NEW.costo_total;
    ELSIF TG_OP = 'UPDATE' THEN
        v_tipo_op      := 'ANULACION';
        v_datos_ant := row_to_json(OLD);
        v_datos_nue := row_to_json(NEW);
        v_costo        := OLD.costo_total; -- costo afectado es el del
registro original
    END IF;

    INSERT INTO modulo5.auditorias_suministros (
        entidad_afectada,
        tipo_operacion,
        datos_anteriores,
        datos_nuevos,
        id_usuario,
        ip_origen,
        fecha_evento,
        fecha_emision,
        id_activo_biologico,
        costo_afectado,
        clasificacion_registro,
        retencion_aplicable,
        resultado
    ) VALUES (
        'registros_consumo_alimentos',
        v_tipo_op,
        v_datos_ant,
        v_datos_nue,
        COALESCE(NEW.id_usuario, OLD.id_usuario),
        NULL, -- IP gestionada por sesión de
backend
        NOW(),
        NOW(), -- timestamp_emision en BD
        COALESCE(NEW.id_activo_biologico, OLD.id_activo_biologico),
        v_costo,
        'NIC41', -- Clasificación per RF-80
catálogo
        '5 años',
        'EXITOSO'
    );

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

```

```
CREATE TRIGGER trg_auditoria_consumo_alimento
AFTER INSERT OR UPDATE ON modulo5.registros_consumo_alimentos
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_auditoria_consumo_alimento();
```

TGR-M05-12: trg_auditoria_medicamento

Descripción	Registra automáticamente en auditorias_suministros todo INSERT o UPDATE sobre registros_medicamentos. Al igual que TGR-M05-11, ahora puebla los campos extendidos de RF-80 que el nuevo esquema de auditorias_suministros expone. AFTER INSERT OR UPDATE sobre registros_medicamentos. Puebla campos base más los extendidos de RF-80, incluyendo costo_afectado desde costo_total_medicamento del registro.
Aplicado a	RF-76 — Registro de Aplicación de Medicamentos / RF-80 — Auditoría y Trazabilidad
Restricción origen RF	<i>RF-76, Criterios CA-6:</i> "Toda operación de registro queda en auditoría con usuario, timestamp y detalle clínico." <i>RF-80, Restricciones, punto 8:</i> "Campos mínimos obligatorios para SUMINISTRO_REGISTRADO/CORREGIDO: id_activo_biologico, id_ciclo_productivo, costo_afectado, origen_precio, id_usuario." <i>RF-76, Requerimientos no funcionales — Trazabilidad:</i> "El historial de tratamientos debe ser consultable con todos los detalles del medicamento, dosis, vía y responsable."

Justificación	Misma razón que TGR-M05-11. Los nuevos campos extendidos de auditorias_suministros deben poblarse desde el trigger para garantizar la completitud del registro de auditoría conforme a RF-80.
----------------------	--

```

CREATE OR REPLACE FUNCTION modulo5.fn_trg_auditoria_medicamento()
RETURNS TRIGGER AS $$
DECLARE
    v_tipo_op    modulo5.enum_auditoria_suministro_tipo_operacion;
    v_datos_ant  json := NULL;
    v_datos_nue  json := NULL;
    v_costo      numeric(16,4) := NULL;
BEGIN
    IF TG_OP = 'INSERT' THEN
        v_tipo_op    := 'REGISTRO';
        v_datos_nue := row_to_json(NEW);
        v_costo      := NEW.costo_total_medicamento;
    ELSIF TG_OP = 'UPDATE' THEN
        v_tipo_op    := 'ANULACION';
        v_datos_ant := row_to_json(OLD);
        v_datos_nue := row_to_json(NEW);
        v_costo      := OLD.costo_total_medicamento; -- costo del
registro original
    END IF;

    INSERT INTO modulo5.auditorias_suministros (
        entidad_afectada,
        tipo_operacion,
        datos_anteriores,
        datos_nuevos,
        id_usuario,
        ip_origen,
        fecha_evento,
        fecha_emision,
        id_activo_biologico,
        costo_afectado,
        clasificacion_registro,
        retencion_aplicable,
        resultado
    ) VALUES (
        'registros_medicamentos',
        v_tipo_op,
        v_datos_ant,

```

```

        v_datos_nue,
        COALESCE(NEW.id_usuario, OLD.id_usuario),
        NULL,                                -- IP gestionada por sesión de
backend
        NOW(),
        NOW(),                                -- timestamp_emision en BD
        COALESCE(NEW.id_activo_biologico, OLD.id_activo_biologico),
        v_costo,
        'NIC41',                             -- Clasificación per RF-80
catálogo
        '5 años',
        'EXITOSO'
    );

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auditoria_medicamento
AFTER INSERT OR UPDATE ON modulo5.registros_medicamentos
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_auditoria_medicamento();

```

TGR-M05-13:

trg_auditoria_costos_productivos

Descripción	Registra automáticamente en auditorias_suministros todo INSERT sobre costos_productivos, garantizando trazabilidad de cada acumulación de inversión. Se dispara en AFTER INSERT sobre costos_productivos. Inserta un registro en auditorias_suministros con tipo operación 'ACUMULACION' y los datos del nuevo registro.
Aplicado a	RF-78 — Acumulación de Inversión / RF-80 — Auditoría
Restricción origen RF	<i>RF-78, Postcondiciones, punto 4: "La operación queda registrada en la bitácora de RF-80/M12." RF-80, Catálogo de eventos:</i>

	"RF-78 produce: SUMINISTRO_REGISTRADO, ACUMULADO_ACTUALIZADO, CICLO_CONSOLIDADO." <i>RF-78,</i> <i>Criterios CA-7:</i> "El cálculo y su resultado quedan registrados en auditoría con usuario, fecha, id_activo y resultado."
Justificación	Cada inserción en costos productivos representa un evento contable NIC 41. Su registro en auditoría debe ser atómico con la inserción misma, no dependiente de una llamada posterior del backend.

```

CREATE OR REPLACE FUNCTION modulo5.fn_auditoria_costos_productivos()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO modulo5.auditorias_suministros (
        entidad_afectada,
        tipo_operacion,
        datos_anteriores,
        datos_nuevos,
        id_usuario,
        ip_origen,
        fecha_evento,
        resultado
    ) VALUES (
        'costos_productivos',
        'ACUMULACION',
        NULL,
        row_to_json(NEW),
        NEW.id_usuario,
        NULL,
        NOW(),
        'EXITOSO'
    );

    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_auditoria_costos_productivos
AFTER INSERT ON modulo5.costos_productivos
FOR EACH ROW

```

```
EXECUTE FUNCTION modulo5.fn_auditoria_costos_productivos();
```

TGR-M05-14:

trg_proteccion_update_costos_productivos

Descripción	Bloquea cualquier UPDATE directo sobre registros de costos_productivos, forzando que las correcciones se realicen únicamente mediante nuevos registros de ajuste. Se dispara en BEFORE UPDATE sobre costos_productivos. Siempre lanza una excepción indicando que se debe usar un registro de corrección.
Aplicado a	RF-78 — Acumulación de Inversión
Restricción origen RF	<i>RF-78, Restricciones, punto 6:</i> "Los registros de suministros son append-only. Las correcciones generan un registro de ajuste (tipo_operacion = CORRECCION) con referencia al registro original (id_registro_original) y motivo_correccion obligatorio. No se permiten DELETE ni UPDATE sobre registros históricos." <i>RF-78, Restricciones, punto 2:</i> "Una vez cerrado el ciclo por RF-41, el monto_inversion_total_ciclo es inmutable."
Justificación	La política append-only del acumulado de inversión es un requisito NIC 41. Ningún UPDATE sobre registros históricos debe ser posible, sin excepción.

```
CREATE OR REPLACE FUNCTION
modulo5.fn_proteccion_update_costos_productivos()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    RAISE EXCEPTION
```

```

        'OPERACION_NO_PERMITIDA: Los registros de costos productivos son
inmutables (append-only, NIC 41). No se permite modificar el registro %.
Para realizar una corrección, inserte un nuevo registro con
tipo_operacion = CORRECCION referenciando el id del registro original.',
        OLD.id_costo_productivo
        USING ERRCODE = 'P0001';
END;
$$;

CREATE TRIGGER trg_proteccion_update_costos_productivos
    BEFORE UPDATE ON modulo5.costos_productivos
    FOR EACH ROW
    EXECUTE FUNCTION modulo5.fn_proteccion_update_costos_productivos();

```

TGR-M05-15:

trg_validacion_estado_tipo_alimento_activativo

Descripción	Valida al momento del INSERT en registros_consumo_alimentos que el tipo de alimento referenciado esté en estado ACTIVO en el catálogo. Se dispara en BEFORE INSERT sobre registros_consumo_alimentos. Verifica que el tipo de alimento esté en estado 'ACTIVO'. Si no, lanza excepción. (Nota: este trigger convive con TGR-M05-03 ya que este verifica el estado y el otro calcula el costo; ambos están en el mismo evento pero hacen cosas distintas. En la implementación final pueden fusionarse en una sola función o mantenerse separados por claridad de responsabilidades.)
Aplicado a	RF-75 — Registro de Consumo de Alimentos
Restricción origen RF	<i>RF-75, Precondiciones, punto 3:</i> "El activo biológico debe estar registrado en el sistema y en estado ACTIVO." (por extensión, los elementos del catálogo también deben estar

	activos para ser usados) <i>RF-75, Proceso, paso 2</i> : "El sistema valida: El activo_biológico_id existe y está en estado ACTIVO."
Justificación	La integridad referencial básica (FK) asegura que el id_tipo_alimento exista, pero no que esté ACTIVO . Este trigger complementa la FK para validar el estado lógico, no solo la existencia física del registro.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_validacion_estado_tipo_alimento_activo()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_estado modulo5.enum_tipo_alimento_estado;
BEGIN
    SELECT estado
    INTO v_estado
    FROM modulo5.tipos_alimentos
    WHERE id_tipo_elemento = NEW.id_tipo_alimento;

    IF NOT FOUND THEN
        RAISE EXCEPTION
            'TIPO_ALIMENTO_NO_ENCONTRADO: El tipo de alimento con id %
no existe en el catálogo.',
            NEW.id_tipo_alimento
            USING ERRCODE = 'P0001';
    END IF;

    IF v_estado != 'ACTIVO' THEN
        RAISE EXCEPTION
            'TIPO_ALIMENTO_INACTIVO: El tipo de alimento con id % está
en estado % y no puede usarse para nuevos registros de consumo.',
            NEW.id_tipo_alimento,
            v_estado
            USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

```

```
CREATE TRIGGER trg_validacion_estado_tipo_alimento_activo
  BEFORE INSERT ON modulo5.registros_consumo_alimentos
  FOR EACH ROW
  EXECUTE FUNCTION
modulo5.fn_validacion_estado_tipo_alimento_activo();
```

TGR-M05-16:

trg_consistencia_fechas_reporte_gastos

Descripción	Valida que en <code>reporte_gastos_acumulados</code> la <code>fecha_inicio_reporte</code> no sea posterior a <code>fecha_fin_reporte</code> y que <code>fecha_fin_reporte</code> no sea futura al momento de la inserción. Se dispara en <code>BEFORE INSERT OR UPDATE</code> sobre <code>reporte_gastos_acumulados</code>. Valida que <code>fecha_inicio_reporte</code> $\leq$ <code>fecha_fin_reporte</code> y que <code>fecha_fin_reporte</code> $\leq$ <code>CURRENT_DATE</code>.
Aplicado a	RF-77 — Reporte de Gastos Acumulados
Restricción origen RF	<i>RF-77, Restricciones, punto 2:</i> "El rango de fechas del reporte no puede ser futuro. <code>fecha_fin_reporte</code> $\leq$ <code>fecha_actual</code> del sistema." <i>RF-77, Flujo alternativo E2:</i> "Rango de fechas inválido (inicio > fin). HTTP 400: Bad Request. Mensaje: 'El rango de fechas es inválido. La fecha de inicio no puede ser posterior a la fecha de fin.'"
Justificación	Las restricciones de rango de fechas en tablas de reportes son integridad de datos que la BD puede garantizar sin ambigüedad, complementando la validación del backend.

```
CREATE OR REPLACE FUNCTION
modulo5.fn_consistencia_fechas_reporte_gastos()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
```

```

BEGIN
    IF NEW.fecha_inicio_reporte > NEW.fecha_fin_reporte THEN
        RAISE EXCEPTION
            'RANGO_FECHAS_INVALIDO: La fecha de inicio del reporte (%)
no puede ser posterior a la fecha de fin (%).',
            NEW.fecha_inicio_reporte,
            NEW.fecha_fin_reporte
        USING ERRCODE = 'P0001';
    END IF;

    IF NEW.fecha_fin_reporte > CURRENT_DATE THEN
        RAISE EXCEPTION
            'FECHA_FUTURA_NO_PERMITIDA: La fecha de fin del reporte (%)
es posterior a la fecha actual del sistema (%). Los reportes no pueden
cubrir períodos futuros.',
            NEW.fecha_fin_reporte,
            CURRENT_DATE
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_consistencia_fechas_reporte_gastos
    BEFORE INSERT OR UPDATE ON modulo5.reporte_gastos_acumulados
    FOR EACH ROW
    EXECUTE FUNCTION modulo5.fn_consistencia_fechas_reporte_gastos();

```

Nota sobre TGR-M05-03 y TGR-M05-15: Ambos se ejecutan en **BEFORE INSERT** sobre la misma tabla. En PostgreSQL los triggers se ejecutan en orden alfabético por nombre dentro del mismo evento, por lo que **trg_calculo_costo_total_consumo_alimento** se ejecuta antes que **trg_validacion_estado_tipo_alimento_activo**. Sin embargo, ambas funciones consultan el estado del tipo de alimento, por lo que en la implementación final del **.sql** puede ser conveniente fusionar ambas en una sola función para evitar la doble consulta, manteniendo la separación conceptual solo a nivel de documentación.

TGR-M05-17:

trg_inmutabilidad_acumulado_ciclo_cerrado

Descripción	Impide cualquier UPDATE o DELETE sobre acumulado_ciclo cuando el campo estado del registro es 'CERRADO', garantizando que el acumulado consolidado al cierre del ciclo productivo sea inmutable. Se dispara en BEFORE UPDATE OR DELETE sobre acumulado_ciclo. Si estado = 'CERRADO', lanza excepción bloqueando la operación. Si es UPDATE hacia 'CERRADO' (el cierre mismo), lo permite; solo bloquea modificaciones posteriores al cierre.
Aplicado a	RF-78 — Acumulación de Inversión por Ciclo Productivo
Restricción origen RF	<i>RF-78, Restricciones, punto 2:</i> "Una vez cerrado el ciclo por RF-41, el monto_inversion_total_ciclo es inmutable. No se admiten nuevos registros ni modificaciones sobre ese ciclo." <i>RF-78, Criterios CA-4:</i> "Dado un ciclo cerrado por RF-41, cuando cualquier proceso intenta modificar monto_inversion_total_ciclo, entonces el sistema rechaza la operación y registra el intento en RF-80 como evento de seguridad."
Justificación	El acumulado consolidado es el dato de referencia para valoración NIC 41 y auditorías del ICA/UPRA. Ningún proceso—incluido el backend, migraciones o scripts de corrección— debe poder alterar este valor una vez cerrado. Es la única garantía técnica de inmutabilidad real.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_trg_acumulado_ciclo_cerrado_inmutable()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        IF OLD.estado = 'CERRADO' THEN
            RAISE EXCEPTION
                'OPERACION_NO_PERMITIDA: El acumulado del ciclo

```

```

productivo [%] '
    'está CERRADO y es inmutable bajo política NIC 41. No se
permite '
    'eliminar registros de ciclos cerrados. (RF-78
Restricción 2)',
    OLD.id_ciclo_productivo
    USING ERRCODE = 'P0001';
END IF;
END IF;

IF TG_OP = 'UPDATE' THEN
    -- Bloquear modificaciones sobre un ciclo ya CERRADO
    IF OLD.estado = 'CERRADO' THEN
        RAISE EXCEPTION
        'OPERACION_NO_PERMITIDA: El acumulado del ciclo
productivo [%] '
        'está CERRADO e inmutable. No se permite modificar el
acumulado '
        'de inversión de un ciclo cerrado. (RF-78 Restricción 2
/ CA-4)',
        OLD.id_ciclo_productivo
        USING ERRCODE = 'P0001';
    END IF;
    -- Bloquear reversión del estado CERRADO a cualquier otro
    IF NEW.estado IS DISTINCT FROM 'CERRADO'
    AND OLD.estado = 'CERRADO' THEN
        RAISE EXCEPTION
        'OPERACION_NO_PERMITIDA: No se puede revertir el estado
CERRADO '
        'del acumulado del ciclo productivo [%]. '
        'El cierre es definitivo e irreversible. (RF-78
Restricción 2)',
        OLD.id_ciclo_productivo
        USING ERRCODE = 'P0001';
    END IF;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_acumulado_ciclo_cerrado_inmutable
BEFORE UPDATE OR DELETE ON modulo5.acumulado_ciclo
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_acumulado_ciclo_cerrado_inmutable();

```


TGR-M05-18:

trg_acumulado_total_no_negativo

Descripción	Valida que el campo <code>acumulado_total_ciclo</code> en <code>acumulado_ciclo</code> nunca quede con valor negativo tras un UPDATE, incluyendo los registros de corrección. Se dispara en <code>BEFORE UPDATE</code> sobre <code>acumulado_ciclo</code>. Si <code>NEW.acumulado_total_ciclo < 0</code>, lanza excepción impidiendo la actualización.
Aplicado a	RF-78 — Acumulación de Inversión por Ciclo Productivo
Restricción origen RF	<i>RF-78, Restricciones, punto 12:</i> "El sistema valida que <code>acumulado_total_ciclo</code> no quede negativo tras la corrección (HTTP 422 si ocurre)." <i>RF-78, Requerimientos no funcionales RNF-09:</i> "El sistema acepta correcciones con <code>costo_registro < 0</code> como ajuste compensatorio válido, con la restricción de que <code>acumulado_total_ciclo</code> resultante no puede ser negativo."
Justificación	El invariante de que el acumulado nunca sea negativo es una restricción de integridad financiera que la BD puede y debe hacer cumplir independientemente de quién realice el UPDATE. El backend puede tener bugs; la BD no.

```
CREATE OR REPLACE FUNCTION modulo5.fn_trg_acumulado_total_no_negativo()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.acumulado_total_ciclo < 0 THEN
        RAISE EXCEPTION
            'ACUMULADO_NEGATIVO_INVALIDO: El acumulado_total_ciclo del
ciclo '
            'productivo [%] no puede ser negativo. Valor resultante:
[%]. '
```

```

        'Verifique el registro de corrección antes de continuar. '
        '(RF-78 Restricción 12 / RNF-09)',
        NEW.id_ciclo_productivo,
        NEW.acumulado_total_ciclo
    USING ERRCODE = 'P0001';
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_acumulado_total_no_negativo
BEFORE UPDATE ON modulo5.acumulado_ciclo
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_acumulado_total_no_negativo();

```

TGR-M05-19:

trg_registro_suministro_append_only

Descripción	Bloquea cualquier operación UPDATE o DELETE sobre registro_suministro, garantizando la política append-only de los registros de suministros del ciclo productivo. Se dispara en BEFORE UPDATE OR DELETE sobre registro_suministro. Siempre lanza excepción en ambos casos.
Aplicado a	RF-78 — Acumulación de Inversión por Ciclo Productivo
Restricción origen RF	<i>RF-78, Restricciones, punto 6:</i> "Los registros de suministros son append-only. Las correcciones generan un registro de ajuste (tipo_operacion = CORRECCION) con referencia al registro original (id_registro_original) y motivo_correccion obligatorio. No se permiten DELETE ni UPDATE sobre registros históricos." <i>RF-78, Requerimientos no funcionales RNF-03:</i> "Registros append-only. Correcciones

	mediante nuevo registro de ajuste con referencia y motivo. Sin DELETE ni UPDATE sobre registros históricos a nivel de base de datos."
Justificación	Esta es precisamente la restricción que RNF-03 exige "a nivel de base de datos". La tabla registro_suministro es la fuente de verdad del acumulado de inversión. Su inmutabilidad debe ser garantizada por el motor, no solo por la aplicación.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_trg_registro_suministro_append_only()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            'OPERACION_NO_PERMITIDA: Los registros de suministros son '
            'append-only (NIC 41 / RF-78 RNF-03). No se permite eliminar
            '
            'el registro con id [%]. Para corregir un valor, inserte un
nuevo '
            'registro con tipo_operacion = CORRECCION referenciando este
id.',
            OLD.id_registro_suministro
        USING ERRCODE = 'P0001';
    END IF;

    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            'OPERACION_NO_PERMITIDA: Los registros de suministros son '
            'append-only (NIC 41 / RF-78 RNF-03). No se permite
modificar '
            'el registro con id [%]. Para corregir un valor, inserte un
nuevo '
            'registro con tipo_operacion = CORRECCION referenciando este
id.',
            OLD.id_registro_suministro
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

```

```
CREATE TRIGGER trg_registro_suministro_append_only
BEFORE UPDATE OR DELETE ON modulo5.registro_suministro
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_registro_suministro_append_only();
```

TGR-M05-20:

trg_registro_suministro_costo_positivo

Descripción	Valida que el campo <code>costo_registro</code> en <code>registro_suministro</code> sea mayor a cero en INSERT, y que <code>cantidad</code> y <code>precio_unitario_resuelto</code> sean también positivos. Se dispara en <code>BEFORE INSERT</code> sobre <code>registro_suministro</code>. Valida que <code>cantidad > 0</code>, <code>precio_unitario_resuelto > 0</code> y que si <code>tipo_operacion = 'REGISTRO'</code> (no corrección), el <code>costo_registro > 0</code>.
Aplicado a	RF-78 — Acumulación de Inversión por Ciclo Productivo
Restricción origen RF	<i>RF-78, Proceso Fase 6:</i> "costo_registro = cantidad × precio_unitario_resuelto. Resultado debe ser > 0. Si la fórmula produce ≤ 0 → rechazo con HTTP 422." <i>RF-78, Entradas — cantidad:</i> "Valor estrictamente > 0. No se admiten cantidades cero ni negativas." <i>RF-78, Entradas — precio_unitario_m40:</i> "Valor > 0."
Justificación	El <code>costo_registro</code> es el componente que alimenta directamente el <code>acumulado_total_ciclo</code>. Un valor de costo cero o negativo en INSERT (que no sea corrección) corrompería el acumulado. La BD es la última defensa antes de que un dato inválido contamine el acumulado NIC 41. Las correcciones negativas son legítimas

	solo cuando <code>tipo_operacion = 'CORRECCION'</code> .
--	--

```
CREATE OR REPLACE FUNCTION
modulo5.fn_trg_registro_suministro_costo_positivo()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.cantidad <= 0 THEN
        RAISE EXCEPTION
            'CANTIDAD_INVALIDA: La cantidad del suministro debe ser
mayor a cero. '
            'Valor recibido: [%]. (RF-78 Entradas -- cantidad)',
            NEW.cantidad
        USING ERRCODE = 'P0001';
    END IF;

    IF NEW.precio_unitario_resuelto <= 0 THEN
        RAISE EXCEPTION
            'PRECIO_INVALIDO: El precio_unitario_resuelto debe ser mayor
a cero. '
            'Valor recibido: [%]. (RF-78 Entradas --
precio_unitario_m40)',
            NEW.precio_unitario_resuelto
        USING ERRCODE = 'P0001';
    END IF;

    -- Para registros normales (no correcciones), el costo debe ser
positivo
    -- Las correcciones pueden tener costo negativo como ajuste
compensatorio
    IF NEW.tipo_operacion = 'REGISTRO' AND NEW.costo_registro <= 0 THEN
        RAISE EXCEPTION
            'COSTO_INVALIDO: El costo_registro debe ser mayor a cero
para '
            'registros de tipo REGISTRO. Valor resultante: [%]. '
            '(RF-78 Proceso Fase 6)',
            NEW.costo_registro
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_registro_suministro_costo_positivo
```

```
BEFORE INSERT ON modulo5.registro_suministro
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_registro_suministro_costo_positivo();
```

TGR-M05-21:

trg_registro_suministro_correccion_req uiere_original

Descripción	Valida que todo registro con <code>tipo_operacion = 'CORRECCION'</code> en <code>registro_suministro</code> tenga obligatoriamente los campos <code>id_registro_original</code> y <code>motivo_correccion</code> con contenido. Se dispara en <code>BEFORE INSERT</code> sobre <code>registro_suministro</code>. Si <code>tipo_operacion = 'CORRECCION'</code>, valida que <code>id_registro_original</code> no sea NULL y que <code>motivo_correccion</code> no sea NULL ni vacío.
Aplicado a	RF-78 — Acumulación de Inversión por Ciclo Productivo
Restricción origen RF	<i>RF-78, Restricciones, punto 6:</i> "Las correcciones generan un registro de ajuste (<code>tipo_operacion = CORRECCION</code>) con referencia al registro original (<code>id_registro_original</code>) y <code>motivo_correccion</code> obligatorio." <i>RF-78, Flujo alterno E7:</i> "<code>tipo_operacion = CORRECCION</code> pero <code>id_registro_original</code> está ausente o no existe → HTTP 422: 'Las correcciones requieren <code>id_registro_original</code> válido.'" <i>RF-78, Criterios CA-9:</i> "Dado <code>tipo_operacion = CORRECCION</code> sin <code>id_registro_original</code> válido, cuando se envía la solicitud, entonces el sistema retorna HTTP 422 sin procesar ninguna persistencia."
Justificación	La trazabilidad de correcciones es un

	requisito de auditoría NIC 41. Un registro de corrección sin referencia al original es un dato huérfano que rompe la cadena de custodia. La BD garantiza que nunca exista tal dato huérfano independientemente del canal de inserción.
--	--

```

CREATE OR REPLACE FUNCTION modulo5.fn_trg_correccion_requiere_original()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.tipo_operacion = 'CORRECCION' THEN

        IF NEW.id_registro_original IS NULL THEN
            RAISE EXCEPTION
                'CORRECCION_SIN_REFERENCIA: Los registros de tipo
CORRECCION '
                'requieren el campo id_registro_original. No se permite
una '
                'corrección sin referencia al registro que se ajusta. '
                '(RF-78 Restricción 6 / Flujo alterno E7)',
            USING ERRCODE = 'P0001';
        END IF;

        IF NEW.motivo_correccion IS NULL
            OR trim(NEW.motivo_correccion) = '' THEN
            RAISE EXCEPTION
                'MOTIVO_CORRECCION_REQUERIDO: Los registros de tipo
CORRECCION '
                'requieren el campo motivo_correccion con contenido. '
                'Describa la razón del ajuste antes de continuar. '
                '(RF-78 Restricción 6)',
            USING ERRCODE = 'P0001';
        END IF;

    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_correccion_requiere_original
BEFORE INSERT ON modulo5.registro_suministro
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_correccion_requiere_original();

```

TGR-M05-22:

trg_provision_nic41_append_only

Descripción	Bloquea cualquier UPDATE o DELETE sobre provision_nic41, garantizando la inmutabilidad de los reportes consolidados de valoración NIC 41 una vez generados. Se dispara en BEFORE UPDATE OR DELETE sobre provision_nic41. Siempre lanza excepción. Las correcciones se materializan como nuevas filas con version_reporte incrementado, nunca como modificaciones de la fila existente.
Aplicado a	RF-79 — Provisión de Datos de Costos para Valoración NIC 41
Restricción origen RF	<i>RF-79, Restricciones, punto 3:</i> "Los reportes consolidados son inmutables una vez generados. Correcciones requieren nuevo reporte con version_reporte = N+1 y referencia al id_reporte anterior (id_reporte_anterior)." <i>RF-79, Criterios CA-7:</i> "Dado un reporte consolidado persistido, cuando cualquier proceso intenta modificar o eliminar sus campos, entonces el sistema rechaza la operación y registra el intento en RF-80."
Justificación	Los reportes consolidados son artefactos formales de valoración NIC 41 que pueden ser auditados en cualquier momento posterior. Su inmutabilidad no puede depender únicamente de controles de aplicación; debe estar garantizada al nivel más bajo posible.

```
CREATE OR REPLACE FUNCTION modulo5.fn_trg_provision_nic41_inmutable()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF TG_OP = 'DELETE' THEN  
        RAISE EXCEPTION  
            'OPERACION_NO_PERMITIDA: Los reportes de provisión NIC 41
```



```

son '
        'inmutables y no pueden eliminarse. El reporte con id [%]
debe '
        'conservarse como evidencia de valoración. Las correcciones
se '
        'realizan mediante un nuevo reporte con version_reporte =
N+1. '
        '(RF-79 Restricción 3 / CA-7)',
        OLD.id_provision
    USING ERRCODE = 'P0001';
END IF;

IF TG_OP = 'UPDATE' THEN
    RAISE EXCEPTION
        'OPERACION_NO_PERMITIDA: Los reportes de provisión NIC 41
son '
        'inmutables y no pueden modificarse. El reporte con id [%] '
        'versión [%] está protegido. Para corregirlo, genere un
nuevo '
        'reporte con version_reporte = [%] y referencia
id_reporte_anterior. '
        '(RF-79 Restricción 3 / CA-7)',
        OLD.id_provision,
        OLD.version_reporte,
        OLD.version_reporte + 1
    USING ERRCODE = 'P0001';
END IF;

RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_provision_nic41_inmutable
BEFORE UPDATE OR DELETE ON modulo5.provision_nic41
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_provision_nic41_inmutable();

```

TGR-M05-23:

trg_provision_nic41_version_correlativ
a

Descripción	Valida que el version_reporte de un nuevo reporte consolidado en provision_nic41 sea exactamente version_anterior + 1 cuando se proporciona id_reporte_anterior, garantizando la cadena de versionado sin saltos. Se dispara en BEFORE INSERT sobre provision_nic41. Si id_reporte_anterior no es NULL, consulta la versión del reporte anterior y valida que NEW.version_reporte = version_anterior + 1.
Aplicado a	RF-79 — Provisión de Datos de Costos para Valoración NIC 41
Restricción origen RF	<i>RF-79, Restricciones, punto 3:</i> "Correcciones requieren nuevo reporte con version_reporte = N+1 y referencia al id_reporte anterior (id_reporte_anterior)." <i>RF-79, Criterios CA-8:</i> "Dado un reporte corregido con version_reporte = N+1, cuando el auditor consulta el reporte, entonces debe encontrar id_reporte_anterior que referencie al reporte de versión N, preservando la cadena de correcciones auditables."
Justificación	La cadena de versionado auditada es un requisito de trazabilidad NIC 41. Un salto de versión (p.ej. de v1 a v3) o una referencia al reporte anterior incorrecto rompe la cadena de custodia. La BD puede verificar esta coherencia de forma determinística.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_trg_provision_version_correlativa()
RETURNS TRIGGER AS $$
DECLARE
    v_version_anterior integer;
BEGIN
    -- Solo aplica cuando se referencia un reporte anterior (corrección)
    IF NEW.id_reporte_anterior IS NOT NULL THEN

        SELECT version_reporte

```

```

        INTO v_version_anterior
        FROM modulo5.provision_nic41
        WHERE id_provision = NEW.id_reporte_anterior;

        IF NOT FOUND THEN
            RAISE EXCEPTION
                'REPORTE_ANTERIOR_NO_ENCONTRADO: El reporte referenciado
en '
                'id_reporte_anterior [%] no existe en provision_nic41. '
                'Verifique el identificador antes de continuar. '
                '(RF-79 Restricción 3)',
                NEW.id_reporte_anterior
            USING ERRCODE = 'P0001';
        END IF;

        IF NEW.version_reporte != (v_version_anterior + 1) THEN
            RAISE EXCEPTION
                'VERSION_NO_CORRELATIVA: El version_reporte [%] no es '
                'correlativo al reporte anterior (versión [%]). Se
esperaba '
                'version_reporte = [%]. La cadena de versiones debe ser
'
                'consecutiva sin saltos. (RF-79 CA-8)',
                NEW.version_reporte,
                v_version_anterior,
                v_version_anterior + 1
            USING ERRCODE = 'P0001';
        END IF;

    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_provision_version_correlativa
BEFORE INSERT ON modulo5.provision_nic41
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_provision_version_correlativa();

```

TGR-M05-24:

trg_resultado_ica_precision_decimal

Descripción	Valida que el campo ca_calculado en resultado_ica se persista con exactamente 4 decimales de precisión, y que data_quality_score esté en el rango permitido de 0 a 100 en incrementos de 25. Se dispara en BEFORE INSERT OR UPDATE sobre resultado_ica. Valida que data_quality_score, cuando está presente, sea uno de los valores permitidos: 0, 25, 50, 75, 100.
Aplicado a	RF-74 — Cálculo de Conversión Alimenticia
Restricción origen RF	<i>RF-74, Restricciones:</i> "El resultado del CA se almacena con precisión de 4 decimales (ejemplo: 2.3456 kg/kg)." <i>RF-74, Restricciones — data_quality_score:</i> "campo data_quality_score (integer, 0–100) calculado como score = (datos_críticos_presentes / 4) × 100. Valores permitidos: 0, 25, 50, 75, 100."
Justificación	El tipo numeric(10,4) de PostgreSQL ya garantiza la precisión en almacenamiento. Sin embargo, el rango de data_quality_score (solo valores 0, 25, 50, 75, 100) no puede expresarse con un constraint CHECK simple sin una lista explícita; un trigger es más claro y mantenible.

```

CREATE OR REPLACE FUNCTION modulo5.fn_trg_resultado_ica_quality_score()
RETURNS TRIGGER AS $$
BEGIN
    -- Validar data_quality_score solo cuando está presente
    IF NEW.data_quality_score IS NOT NULL THEN
        IF NEW.data_quality_score NOT IN (0, 25, 50, 75, 100) THEN
            RAISE EXCEPTION
                'DATA_QUALITY_SCORE_INVALIDO: El data_quality_score [%]
no es '
                'un valor permitido. Los valores válidos son: 0, 25, 50,
75, 100 '
                'según la fórmula score = (datos_críticos_presentes / 4)
× 100. '
        
```

```

        '(RF-74 Restricciones -- data_quality_score)',
        NEW.data_quality_score
        USING ERRCODE = 'P0001';
    END IF;
END IF;

-- Validar que ca_calculado no sea negativo cuando está presente
IF NEW.ca_calculado IS NOT NULL AND NEW.ca_calculado < 0 THEN
    RAISE EXCEPTION
        'CA_CALCULADO_INVALIDO: El ca_calculado no puede ser
negativo. '
        'Valor recibido: [%]. El CA es la razón alimento/ganancia de
peso '
        'y siempre debe ser positivo. (RF-74 Restricciones)',
        NEW.ca_calculado
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_resultado_ica_quality_score
BEFORE INSERT OR UPDATE ON modulo5.resultado_ica
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_resultado_ica_quality_score();

```

TGR-M05-25:

trg_resultado_ica_append_only

Descripción	Bloquea la eliminación física de registros de resultado_ica, preservando el historial completo de cálculos del CA por activo y período. Se dispara en BEFORE DELETE sobre resultado_ica. Siempre lanza excepción.
Aplicado a	RF-74 — Cálculo de Conversión Alimenticia

Restricción origen RF	<i>RF-74, Restricciones:</i> "Si existe un cálculo previo para el mismo período, el nuevo resultado reemplaza al anterior, preservando el histórico en la tabla historial_ica." <i>RF-74, Criterios CA-5:</i> "El sistema preserva el historial de todos los cálculos anteriores del CA por activo y período." <i>RF-74, Postcondiciones punto 2:</i> "El historial de CA del activo está actualizado y disponible para el módulo de valoración financiera (M06)."
Justificación	La preservación del historial de CA es un requisito explícito para M06 (valoración NIC 41). Ningún proceso debe poder borrar este historial, ya que es el registro de eficiencia productiva a lo largo del tiempo del activo.

```

CREATE OR REPLACE FUNCTION modulo5.fn_trg_resultado_ica_no_delete()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION
        'OPERACION_NO_PERMITIDA: Los registros de resultado ICA son '
        'append-only y no pueden eliminarse. El registro con id [%] del
        '
        'activo [%] período [%] debe conservarse en el historial de CA '
        'para trazabilidad y valoración NIC 41 (M06). '
        '(RF-74 CA-5 / Postcondición 2)',
        OLD.id_resultado_ica,
        OLD.id_activo_biologico,
        OLD.periodo_evaluacion
    USING ERRCODE = 'P0001';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_resultado_ica_no_delete
BEFORE DELETE ON modulo5.resultado_ica
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_resultado_ica_no_delete();

```

TGR-M05-26:

trg_auditoria_registro_suministro

Descripción	Registra automáticamente en auditorias_suministros cada INSERT sobre registro_suministro, incluyendo los campos extendidos de RF-80 (id_activo_biologico, id_ciclo_productivo, costo_afectado, origen_precio). Se dispara en AFTER INSERT sobre registro_suministro. Determina el tipo de operación según tipo_operacion del registro y persiste en auditorias_suministros con los campos extendidos de RF-80.
Aplicado a	RF-78 — Acumulación de Inversión / RF-80 — Auditoría y Trazabilidad
Restricción origen RF	<i>RF-78, Postcondiciones, punto 4:</i> "La operación queda registrada en la bitácora de RF-80/M12." <i>RF-80, Catálogo de eventos:</i> "RF-78 produce: SUMINISTRO_REGISTRADO, SUMINISTRO_CORREGIDO, ACUMULADO_ACTUALIZADO..." <i>RF-80, Restricciones, punto 8:</i> "Campos mínimos obligatorios — SUMINISTRO_REGISTRADO/CORREGIDO: id_registro_suministro, id_tipo_suministro, costo_afectado, origen_precio, id_usuario."
Justificación	La tabla registro_suministro es nueva en este esquema y no estaba cubierta por los triggers de auditoría anteriores (que solo cubrían costos_productivos, registros_consumo_alimentos y registros_medicamentos). Cada INSERT aquí es un evento SUMINISTRO_REGISTRADO o SUMINISTRO_CORREGIDO que RF-80 exige que quede en auditoría.

```
CREATE OR REPLACE FUNCTION
```

```
modulo5.fn_trg_auditoria_registro_suministro()
```

```

RETURNS TRIGGER AS $$
DECLARE
    v_tipo_op modulo5.enum_auditoria_suministro_tipo_operacion;
BEGIN
    -- Determinar tipo de operación según naturaleza del registro
    IF NEW.tipo_operacion = 'CORRECCION' THEN
        v_tipo_op := 'CORRECCION';
    ELSE
        v_tipo_op := 'REGISTRO';
    END IF;

    INSERT INTO modulo5.auditorias_suministros (
        entidad_afectada,
        tipo_operacion,
        datos_anteriores,
        datos_nuevos,
        id_usuario,
        ip_origen,
        fecha_evento,
        fecha_emision,
        id_activo_biologico,
        id_ciclo_productivo,
        costo_afectado,
        origen_precio,
        clasificacion_registro,
        retencion_aplicable,
        resultado
    ) VALUES (
        'registro_suministro',
        v_tipo_op,
        NULL,
        row_to_json(NEW),
        NULL, -- id_usuario: no está en registro_suministro;
        NULL, -- ip_origen: gestionada por sesión de backend
        NOW(),
        NOW(), -- timestamp_emision = timestamp_registro en BD
        NEW.id_activo_biologico,
        NEW.id_ciclo_productivo,
        NEW.costo_registro,
        NEW.origen_precio,
        'NIC41', -- Clasificación fija per RF-80 catálogo
        '5 años',
        'EXITOSO'
    );

```



```
        RETURN NULL;
    END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auditoria_registro_suministro
AFTER INSERT ON modulo5.registro_suministro
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_trg_auditoria_registro_suministro();
```